

M2 Command-Line Interface Design

October 9, 2026

Contents

1. Summary
 2. Background and goal
 3. Design
 4. Implementation plan
 5. Evaluation
 6. Risks
- References

Purpose. This document describes the input parsing, output format, error handling and test method of a command-line interface that lets M1's `convert()` be called from the shell as `python convert.py 100 km mi`, and seeks approval of the approach and the completion criteria before implementation.

1 Summary

M2 adds a single `main()` to `convert.py` so that it can be used from the command line. The conversion arithmetic stays as it was in M1[D1].

- **Input.** `argparse` takes the value, the source unit and the target unit. Only finite numbers are accepted as the value, because a preliminary test showed that `nan` and `inf` pass through unchanged[D3].
- **Output.** The result is printed with 10 significant digits followed by the unit (`62.13711922 mi`). This way a floating-point residue such as `211.99999999999994` appears as `212`.
- **Errors.** All errors follow the `argparse` convention: one usage line and one error line on standard error, then exit code 2.
- **Tests.** 10 tests are added that launch a real process and check the exit code and output.

2 Background and goal

M1 completed `convert(value, src, dst)`. All 127 tests passed and it has 0 imports[D1]. At present it can only be called from inside Python code. The completion criteria for M2 are in Table 5 of the proposal[D2]. Table 1 lists what M2 inherits from M1 and what M2 has to do.

Table 1. What M2 inherits from M1 and what M2 does. [D1, D2]

Item	State after M1	In M2
Conversion	<code>convert()</code> , accuracy tests done	Called as is
Three error types	<code>ValueError</code> with message	Caught, exit code 2
Non-numeric value	Deferred to M2	Rejected during input parsing
Unit notation	Error messages in lowercase (developer decision, 2026-10-09)	Kept as is

3 Design

Figure 1 shows the call flow now that M1 is finished (a) and after M2 (b). Two parts are new: input parsing and output.

(a) Now (after M1)



(b) After M2

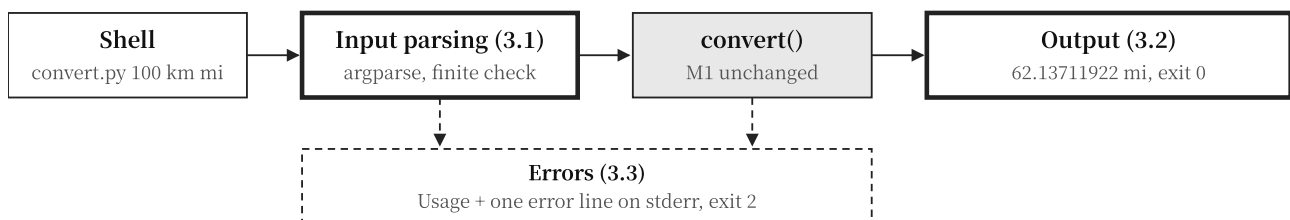


Figure 1. Flow now (a) and after M2 (b). Bold borders mark the new parts, grey marks the parts reused unchanged from M1, and dashed lines are error paths. Errors arise in two places, input parsing and conversion, but end in the same format.

3.1 Input parsing

The interface takes three positional arguments, `value src dst`. `value` goes through a function that turns the string into a number. This function converts with `float()`, checks with `math.isfinite()` that the number is finite, and raises `argparse.ArgumentTypeError` if it is not[R1,

R2]. Because `float()` accepts "nan" and "inf" as numbers, a separate finiteness check is needed. Table 2 shows how `argparse` actually parses each input in Python 3.11.16, checked in advance[D3].

Table 2. How `argparse` parses each input (default float conversion, preliminary check). [D3, R1]

Input	argparse result	In M2
-40 c f	value = -40.0	As is (negative numbers work)
-1.5 c f	value = -1.5	As is
abc km mi	exit 2, "invalid float value: 'abc'"	As is (non-numeric value rejected)
nan km mi	value = nan (passes)	Rejected by finiteness check
inf km mi	value = inf (passes)	Rejected by finiteness check
-1e3 m km	exit 2, "invalid float value: 'm'" (misleading message)	Help text explains using -- (Section 3.3)
-- -1e3 m km	value = -1000.0	As is

3.2 Output

The result is written as one line to standard output in the format `f"{result:.10g} {dst}"`. `.10g` rounds to 10 significant digits, drops unneeded zeros and the decimal point, and switches very large or very small values to exponent notation[R3]. The unit appears exactly as the user typed it (`100 c F` $\rightarrow$ `212 F`). Table 3 shows how actual computed values appear.

Table 3. Output format examples. Checked in Python 3.11.16. [D3]

Computed value	Output	Note
62.13711922373339	62.13711922	10 significant digits
211.99999999999994	212	Floating-point residue disappears
-40.000000000000006	-40	Same
1000000.0	1000000	Unneeded ".0" removed
1.5e+20 / 1e-07	1.5e+20 / 1e-07	Large and small values in exponent notation

3.3 Errors and help

A `ValueError` raised by `convert()` is passed to `parser.error(message)`, the same path `argparse` uses for its own errors (such as a non-numeric value). It writes one usage line and one

`convert.py`: `error`: ... line to standard error and exits with code 2[R1]. The proposal specified a “one-line message”[D2], but **to keep a single error format, a usage line is added in line with the argparse convention**. This way every error has the same shape and there is only one exit code.

The end of the `-h` help text lists the available units (built from the M1 table) and advises putting `--` before negative numbers in exponent notation. From Python 3.14 on, `-1e3` is also recognized as a negative number, so this advice will no longer be needed[R1].

4 Implementation plan

There are no new files. The new dependencies are two standard library modules, `argparse` and `math`. Table 4 lists the changes.

Table 4. Files changed.

File	Change	Reason
<code>convert.py</code>	Finite-number conversion function, <code>main(argv=None)</code> , if <code>__name__ == "__main__"</code> block. <code>import argparse, math</code>	Sections 3.1–3.3
<code>test_convert.py</code>	Add a CLI test group. Real execution via <code>subprocess</code>	Check exit code and output

- ① **Tests first**. Write the CLI tests in Table 5 and confirm that they fail.
- ② **Write `main()`**. Make the tests pass.
- ③ **Full run**. Run everything together with the 7 M1 tests, and run a few commands directly in the shell to check the output by eye.

5 Evaluation

The CLI tests launch a real process with `subprocess.run([sys.executable, "convert.py", ...])` and check standard output, standard error and the exit code[R4]. Tests that only call the function cannot check the exit code or how the shell arguments are parsed. Table 5 lists the test cases and the completion criteria.

Table 5. CLI tests and completion criteria. Includes the M2 criteria from Tables 4 and 5 of the proposal. [D2]

Group	Input → expected	Target
Normal output	100 km mi → 62.13711922 mi, -40 C F → -40 F, -- -1e3 m km → -1 km; all exit 0	3 / 3
Errors	Unknown unit (1 km xx), kind mismatch (1 km kg), non-numeric (abc km mi), below 0 K (-300 c f), nan, inf; all exit 2, empty standard output, reason on standard error	6 / 6
Help	-h → exit 0, includes unit list and -- advice	1 / 1
Regression	The 7 M1 tests	7 / 7
Dependencies	convert.py imports	argparse, math only

6 Risks

Table 6 lists problems that may arise in M2 and how to detect them.

Table 6. Risks and how to detect them.

Risk	How to detect	Response
Misleading error for -1e3	Already confirmed in Table 2 (“invalid float value: 'm'”)	Handled by the help text. A workaround that parses the arguments by hand is not used because it adds code. Resolved automatically on 3.14 and later[R1].
convert.py not found depending on where the tests run	Running the tests from another folder makes the CLI tests fail.	Build the path from the location of the test file.
Windows console encoding	The output contains only digits and ASCII unit symbols, so no effect is expected (not verified).	Do not print symbols such as °.

References

- [R1] Python Software Foundation. argparse — Parser for command-line options, arguments and subcommands (“Arguments containing -”, ArgumentParser.error, type).
<https://docs.python.org/3/library/argparse.html>
- [R2] Python Software Foundation. math.isfinite.
<https://docs.python.org/3/library/math.html#math.isfinite>

- [R3] Python Software Foundation. Format Specification Mini-Language (“g” presentation type).
<https://docs.python.org/3/library/string.html#format-specification-mini-language>
- [R4] Python Software Foundation. subprocess.run.
<https://docs.python.org/3/library/subprocess.html#subprocess.run>
- [D1] M1 Conversion Core Result (docs/reports/unit-converter-cli/M1-result.pdf), approved 2026-10-09.
- [D2] Unit Conversion CLI Proposal (docs/reports/unit-converter-cli/00-proposal.pdf), approved 2026-10-09.
- [D3] Preliminary check of argparse parsing and output format, Python 3.11.16, 2026-10-09.